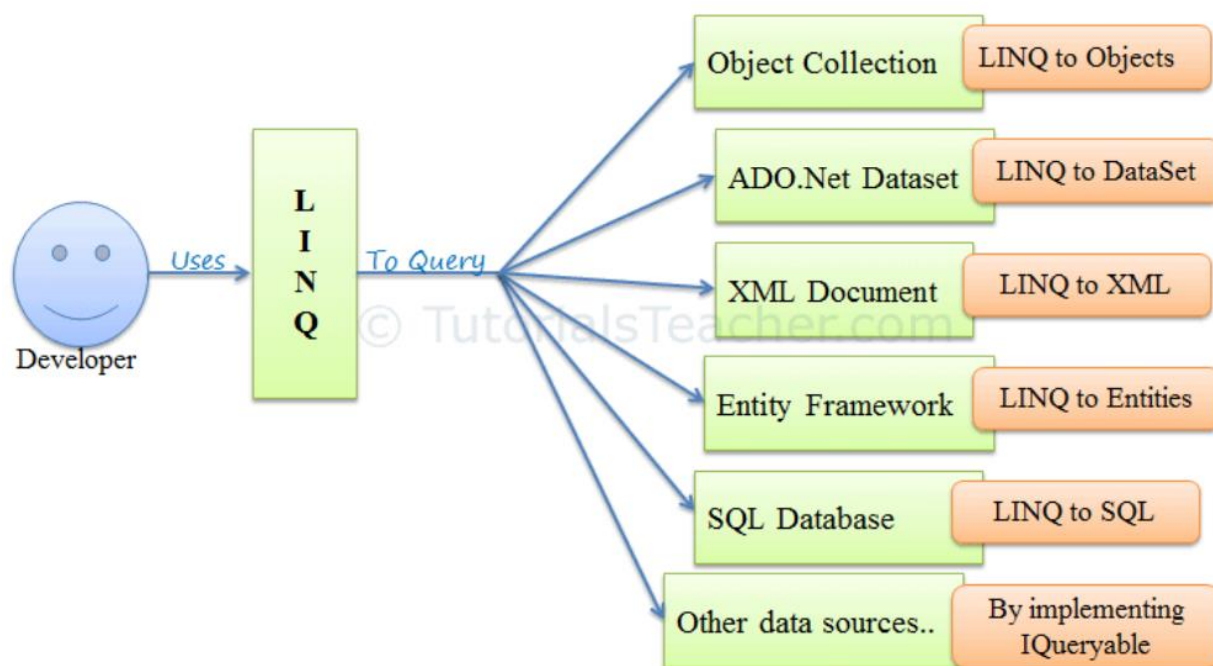


LINQ چیست؟

Query (LINQ) یکپارچه زبان (یک دستور پرس و جو یکنواخت در C# و VB.NET برای بازیابی داده ها از منابع و فرمت های مختلف است. این در C# یا VB یکپارچه شده است، در نتیجه عدم تطابق بین زبان های برنامه نویسی و پایگاه های داده را حذف می کند، و همچنین یک رابط پرس و جو واحد برای انواع مختلف منابع داده ارائه می دهد.

به عنوان مثال، SQL یک زبان پرس و جو ساخت یافته است که برای ذخیره و بازیابی داده ها از پایگاه داده استفاده می شود. به همین ترتیب، LINQ یک دستور ساختار یافته است که در C# و VB.NET برای بازیابی داده ها از انواع مختلف منابع داده مانند مجموعه ها، XML Docs، ADO.Net DataSet، وب سرویس و MS SQL Server و سایر پایگاه های داده ساخته شده است.



کوئری های LINQ نتایج را به صورت اشیا برمی گرداند. این به شما امکان می دهد از رویکرد شی گرا در مجموعه نتایج استفاده کنید و نگران تبدیل فرمت های مختلف نتایج به اشیا نباشید.



مثال زیر یک کوئری ساده LINQ را نشان می دهد که تمام رشته ها را از آرایه ای که حاوی "a" است دریافت می کند.

مثال: LINQ Query to Array

```
// Data source
```

```
string[] names = {"Bill", "Steve", "James", "Mohan"};
```

```
// LINQ Query
```

```
var myLinqQuery = from name in names  
                  where name.Contains('a')  
                  select name;
```

```
// Query execution
```

```
foreach(var name in myLinqQuery)  
    Console.WriteLine(name + " ");
```

در مثال بالا، نام آرایه های رشته ای یک منبع داده است. در زیر یک کوئری LINQ است که به یک متغیر `myLinqQuery` اختصاص داده شده است.

```
from name in names  
where name.Contains('a')  
select name;
```

تا زمانی که پرس و جوی LINQ را اجرا نکنید، به نتیجه نخواهید رسید. پرس و جو LINQ را می توان به روش های مختلف اجرا کرد، در اینجا ما از حلقه `foreach` برای اجرای پرس و جو ذخیره شده در مجموعه `myLinqQuery` استفاده کردیم، بنابراین، هر پرس و جوی LINQ باید به نوعی از منابع داده پرس و جو کند، خواه آرایه، مجموعه، XML یا پایگاه های داده دیگر باشد. پس از نوشتن کوئری LINQ باید آن را اجرا کرد تا نتیجه به دست آید.

در ادامه می آموزید که چرا باید از LINQ استفاده کنیم.

چرا LINQ؟

برای درک اینکه چرا باید از LINQ استفاده کنیم، به چند نمونه نگاه می کنیم. فرض کنید می خواهید لیستی از دانش آموزان را از آرایه ای از اشیاء `Student` پیدا کنید.

قبل از C# ۲.۰، ما مجبور بودیم از یک حلقه `"foreach"` یا `"for"` برای پیمایش مجموعه برای یافتن یک شی خاص استفاده کنیم. به عنوان مثال، ما مجبور شدیم کد زیر را بنویسیم تا همه اشیاء `Student` را از آرایه ای از `Students` که سن آنها بین ۱۲ تا ۲۰ سال است (برای نوجوانان ۱۳ تا ۱۹) پیدا کنیم:

مثال: از حلقه `for` برای یافتن عناصر مجموعه در C# ۱.۰ استفاده می شد

```
class Student
{
    public int StudentID { get; set; }
    public String StudentName { get; set; }
    public int Age { get; set; }
}

class Program
{
    static void Main(string[] args)
```

```

{
    Student[] studentArray = {
        new Student() { StudentID = ۱, StudentName = "Ali", Age = ۱۸ },
        new Student() { StudentID = ۲, StudentName = "Ahmad", Age = ۲۱ },
        new Student() { StudentID = ۳, StudentName = "Reza", Age = ۲۵ },
        new Student() { StudentID = ۴, StudentName = "Sara", Age = ۲۰ },
        new Student() { StudentID = ۵, StudentName = "Atusa", Age = ۳۱ },
        new Student() { StudentID = ۶, StudentName = "parmiss", Age = ۱۷ },
        new Student() { StudentID = ۷, StudentName = "Amir", Age = ۱۹ },
    };

    Student[] students = new Student[۱۰];

    int i = ۰;

    foreach (Student std in studentArray)
    {
        if (std.Age > ۱۲ && std.Age < ۲۰)
        {
            students[i] = std;
            i++;
        }
    }
}

```

استفاده از حلقه for دست و پا گیر است، قابل نگهداری و خواندن نیست ۲.۰ C#. یک [delegate](#) معرفی کرد که می تواند برای مدیریت این نوع سناریوها، استفاده شود. مثال: از Delegates برای یافتن عناصر مجموعه در ۲.۰ C# استفاده کردیم:

```
delegate bool FindStudent(Student std);
```

```
class StudentExtension
```

```

{
    public static Student[] where(Student[] stdArray, FindStudent del)
    {
        int i=0;
        Student[] result = new Student[10];
        foreach (Student std in stdArray)
            if (del(std))
            {
                result[i] = std;
                i++;
            }

        return result;
    }
}

```

```

class Program
{
    static void Main(string[] args)
    {
        Student[] studentArray = {
            new Student() { StudentID = 1, StudentName = "Ali", Age = 18 },
            new Student() { StudentID = 2, StudentName = "Ahmad", Age = 21 },
            new Student() { StudentID = 3, StudentName = "Reza", Age = 25 },
            new Student() { StudentID = 4, StudentName = "Sara", Age = 20 },
            new Student() { StudentID = 5, StudentName = "Atusa", Age = 31 },
            new Student() { StudentID = 6, StudentName = "parmiss", Age = 17 },
            new Student() { StudentID = 7, StudentName = "Amir", Age = 19 },
        };
    }
}

```

```

Student[] students = StudentExtension.where(studentArray, delegate(Student
std){
    return std.Age > ۱۲ && std.Age < ۲۰;
});
}
}
}

```

بنابراین، در نسخه C# ۲.۰ به بعد از مزیت **delegate** در یافتن دانش آموزان با هر معیاری برخوردارید. برای یافتن دانش آموزان با معیارهای مختلف، لازم نیست از حلقه for استفاده کنید. برای مثال، می‌توانید از همان تابع delegate برای پیدا کردن دانش‌آموزی که StudentId او ۵ است یا نامش Atusa است، استفاده کنید، مانند زیر:

```

Student[] students = StudentExtension.where(studentArray, delegate(Student std)
{
    return std.StudentID == ۵;
});

```

//Also, use another criteria using same delegate

```

Student[] students = StudentExtension.where(studentArray, delegate(Student std)
{
    return std.StudentName == "Atusa";
});

```

تیم سی شارپ احساس کردند که هنوز باید کد را فشرده‌تر و خواناتر کنند. بنابراین آنها anonymous type ، expression tree ،lambda expression ،extension method ،query expression را در [سی شارپ ۳/۰ معرفی کردند](#) . شما می‌توانید از این ویژگی‌های C# ۳.۰ استفاده کنید، که بلوک‌های سازنده LINQ برای پرس و جو در انواع مختلف مجموعه و دریافت عنصر(های) به دست آمده در یک عبارت واحد هستند.

مثال زیر نشان می دهد که چگونه می توانید از پرس و جوی LINQ با عبارت lambda برای یافتن دانش آموز خاصی از مجموعه دانش آموز استفاده کنید.

```
Student[] studentArray = {  
    new Student() { StudentID = ۱, StudentName = "John", age = ۱۸ },  
    new Student() { StudentID = ۲, StudentName = "Steve", age = ۲۱ },  
    new Student() { StudentID = ۳, StudentName = "Bill", age = ۲۵ },  
    new Student() { StudentID = ۴, StudentName = "Ram", age = ۲۰ },  
    new Student() { StudentID = ۵, StudentName = "Ron", age = ۳۱ },  
    new Student() { StudentID = ۶, StudentName = "Chris", age = ۱۷ },  
    new Student() { StudentID = ۷, StudentName = "Rob", age = ۱۹ },  
};  
  
// Use LINQ to find teenager students  
Student[] teenAgerStudents = studentArray.Where(s => s.age > ۱۲ && s.age <  
۲۰).ToArray();  
  
// Use LINQ to find first student whose name is Bill  
Student bill = studentArray.Where(s => s.StudentName ==  
"Bill").FirstOrDefault();  
  
// Use LINQ to find student whose StudentID is ۵  
Student student۵ = studentArray.Where(s => s.StudentID ==  
۵).FirstOrDefault();  
}  
}
```

مزایای استفاده از LINQ

- **زبان آشنا:** توسعه دهندگان مجبور نیستند برای هر نوع منبع داده یا قالب داده، یک زبان جستجوی جدید یاد بگیرند.
- **کدنویسی کمتر:** در مقایسه با رویکرد سنتی تر، مقدار کدی را که باید نوشته شود کاهش می دهد.
- **کد قابل خواندن LINQ:** کد را خوانا تر می کند تا توسعه دهندگان دیگر بتوانند به راحتی آن را درک کرده و حفظ کنند.
- **روش استاندارد پرس و جو از چندین منبع داده:** از همان ساختار LINQ می توان برای پرس و جو از چندین منبع داده استفاده کرد.
- **ایمنی زمان کامپایل کوئری ها:** بررسی نوع اشیاء را در زمان کامپایل فراهم می کند.
- **پشتیبانی از IntelliSense: LINQ IntelliSense** را برای مجموعه های عمومی فراهم می کند.
- **شکل دادن به داده ها:** می توانید داده ها را به اشکال مختلف بازایی کنید.